CODEBASE LEARNING GUIDE · BEGINNER TO CONFIDENT

FieldGuard Pro

Field Inspection & Asset Maintenance Application

A guided tour of every concept in the project: where the code starts, how Angular, Ionic and Capacitor fit together, the offline-first sync engine, and how a tap in the UI travels all the way to storage.

Angular 22 · Ionic 9 · Capacitor 8 · RxJS · Signals · TypeScript 6

Based on the repository state of October 2026 · Standalone Angular · Offline-first architecture

Contents

Click any entry to jump to that section.

1. The Big Picture . . . 5

1.1 What does the app do? . . . 5
1.2 The technology stack as layers . . . 5
1.3 One app, two ways to run it . . . 6
1.4 Reading order that works . . . 6

2. Project Anatomy . . . 7

2.1 Top-level layout . . . 7
2.2 Inside src/app (feature-based structure) . . . 7
2.3 The configuration files explained . . . 8

3. Where the Code Begins . . . 9

3.1 The 30-second answer . . . 9
3.2 Step-by-step startup sequence . . . 9
3.3 Reading main.ts line by line . . . 10
3.4 The root component . . . 10
3.5 The routing table (app.routes.ts) . . . 11
3.6 What really wakes up when the Dashboard opens . . . 11

4. TypeScript & Angular Concepts . . . 13

4.1 TypeScript concepts you will meet . . . 13
4.2 Standalone components . . . 13
4.3 Templates: bindings and the new control flow . . . 14
4.4 Dependency Injection (DI) . . . 14
4.5 Signals - the reactive state primitive . . . 14
4.6 RxJS Observables . . . 15
4.7 Component lifecycle hooks . . . 15
4.8 Reactive Forms and conditional validation . . . 16
4.9 Guards, pipes, @Input/@Output, @ViewChild . . . 16

5. Ionic Framework Concepts . . . 17

5.1 What is Ionic? . . . 17
5.2 Anatomy of an Ionic page . . . 17
5.3 Ionic components used in this project . . . 17
5.4 Slots, colours and properties . . . 18

5.5 Navigation: how Ionic differs from plain Angular . . . 18
5.6 Overlay controllers (programmatic UI) . . . 19
5.7 Icons . . . 19
5.8 Theming and global CSS . . . 20
5.9 @ionic/pwa-elements . . . 20

6. Capacitor Concepts . . . 21

6.1 What is Capacitor? . . . 21
6.2 capacitor.config.ts . . . 21
6.3 Plugins: installed vs. actually used . . . 21
6.4 The Camera plugin in this app . . . 22
6.5 The Network plugin in this app . . . 22
6.6 Build and run on a real device . . . 23
6.7 Storage on a device . . . 23

7. Architecture & Design Patterns . . . 24

7.1 Layered view . . . 24
7.2 Repository pattern . . . 24
7.3 Adapter pattern . . . 25
7.4 Offline-first strategy . . . 26

8. Domain Model & State Machines . . . 27

8.1 The Inspection model . . . 27
8.2 Workflow status machine . . . 27
8.3 Sync status and queue item status . . . 28
8.4 Roles and permissions . . . 28
8.5 Seed data . . . 28

9. Core Services Deep Dive . . . 29

9.1 AuthService (core/auth/auth.service.ts) . . . 29
9.2 NetworkService . . . 29
9.3 ToastService & CameraService . . . 30
9.4 SyncInspectionRepository in detail . . . 30
9.5 SyncQueueService . . . 31
9.6 All persisted keys . . . 31

10. Feature Pages Walkthrough . . . 33

10.1 LoginPage (/login) . . . 33
10.2 DashboardPage (/dashboard, authGuard) . . . 33

10.3 InspectionListPage (/inspections) . . . 33
10.4 InspectionFormPage (/inspections/new and /inspections/:id/edit) . . . 34
10.5 InspectionDetailsPage (/inspections/:id) . . . 34
10.6 ApprovalListPage (/approvals, supervisor only) . . . 34

11. Shared Components & Pipe . . . 35

11.1 SyncBadgeComponent () . . . 35
11.2 SignaturePadComponent () . . . 35
11.3 QrScannerModalComponent . . . 35
11.4 ImagePreviewModalComponent . . . 35
11.5 StatusColorPipe ({{ value | statusColor:'type' }}) . . . 36

12. End-to-End Scenarios . . . 37

12.1 First launch and sign-in . . . 37
12.2 Create an inspection with damage and submit online . . . 37
12.3 Submit while offline, then reconnect . . . 37
12.4 Supervisor approves or rejects . . . 38
12.5 Protection scenarios . . . 38

13. Tooling, Build & Testing . . . 40

13.1 npm scripts . . . 40
13.2 Build configurations . . . 40
13.3 Unit testing with Vitest . . . 40
13.4 Debugging tips . . . 41

14. How to Extend the App . . . 42

14.1 Add a new page . . . 42
14.2 Replace localStorage with a real database . . . 42
14.3 Connect a real backend . . . 42
14.4 Add native capabilities . . . 42

15. Honest Notes & Known Limitations . . . 43

16. Learning Path, Exercises & Cheat Sheet . . . 44

16.1 Suggested learning path . . . 44
16.2 Exercises (with hints) . . . 44
16.3 "Where do I find...?" cheat sheet . . . 44

17. Glossary . . . 46

1. The Big Picture

What this app is, which technologies it uses, and how to think about them as layers.

1.1 What does the app do?

FieldGuard Pro is a mobile-first enterprise app for **field engineers** who walk around industrial sites (pumps, turbines, transformers, tanks...) and inspect equipment. A **supervisor** then reviews what was found. The key promise of the app is that it keeps working **without internet** and quietly synchronises when the connection returns.

Capability	Where it lives in the code
Sign in & roles (Field Engineer / Supervisor)	`core/auth/` and `features/auth/login/`
Dashboard with KPIs and recent activity	`features/dashboard/dashboard.page.ts`
Create / edit inspection with validation	`features/inspections/inspection-form/`
Photos (camera or gallery), signature, QR asset pick	`core/services/camera.service.ts`, `shared/components/*`
Offline-first save + automatic sync	`core/repositories/*` and `core/sync/*`
Supervisor approve / reject with audit trail	`features/approvals/`, `inspection-details/`

1.2 The technology stack as layers

Beginners often mix up Angular, Ionic and Capacitor. They are **three different things stacked on top of each other**. Learn the job of each layer and the rest of the codebase becomes much easier to read.



Figure 1.1 - The stack, top (what you write) to bottom (the device).

IN PLAIN ENGLISH

Think of it like building a house. **Angular** is the structure and wiring (how things connect), **Ionic** is the furniture and paint (how it looks and feels like a phone app), and **Capacitor** is the door to the outside world (the phone's camera, network status, files).

1.3 One app, two ways to run it

Mode	How	What happens
Browser (today)	`npm start` -> http://localhost:4200	Angular+Ionic run in Chrome/Edge. Capacitor plugins fall back to web implementations or to code fallbacks.
Native app (next step)	`npx cap add android` / `ios`, then `npx cap sync`	The *same* `www/` build is copied inside a native shell; Capacitor plugins call real native APIs.

GOOD TO KNOW

This repository currently contains **no** `android/` **or** `ios/` **folder**, so only the web mode has been exercised. `capacitor.config.ts` is already in place, so adding a native platform is a one-command step (see Chapter 6).

1.4 Reading order that works

1. Chapters 1-3 give you the mental model and the start-up path (read these first, in order).
2. Chapters 4-6 are concept references for Angular, Ionic and Capacitor - dip in as needed.
3. Chapters 7-9 explain the architecture (repositories, adapters, offline sync) - the intellectual core.
4. Chapters 10-12 walk through each screen and trace complete user journeys step by step.
5. Chapters 13-17 cover tooling & tests, extension recipes, honest limitations, exercises and a glossary.

2. Project Anatomy

Every folder and config file, and what role it plays.

2.1 Top-level layout

Folder map

```
Field Inspection & Asset Maintenance Application POC/
|-- angular.json            <- Angular build/serve/test/lint configuration
|-- capacitor.config.ts     <- Capacitor: appId, appName, webDir ("www")
|-- ionic.config.json       <- Ionic CLI: project type "angular-standalone"
|-- package.json            <- dependencies + npm scripts (start, build, test, lint)
|-- tsconfig*.json          <- TypeScript compiler settings (strict mode ON)
|-- eslint.config.js        <- code-style rules (angular-eslint)
|-- .browserslistrc         <- which browsers the build must support
|-- README.md               <- project summary
|-- www/                    <- OUTPUT of `npm run build` (what Capacitor ships)
|-- node_modules/           <- installed libraries (never edit)
`-- src/                    <- ALL application source code
    |-- index.html          <- the single HTML page (contains <app-root>)
    |-- main.ts             <- JavaScript entry point (bootstraps Angular)
    |-- global.scss         <- global styles + Ionic CSS imports
    |-- theme/variables.scss<- brand colour tokens (--ion-color-*)
    |-- test-setup.ts       <- polyfills for unit tests
    |-- environments/       <- environment.ts / environment.prod.ts
    |-- assets/             <- icons and static files
    `-- app/                <- the application itself (next section)
```

2.2 Inside src/app (feature-based structure)

Feature-based folder structure

```
src/app/
|-- app.component.ts|html   <- root component: <ion-app><ion-router-outlet/></ion-app>
|-- app.routes.ts           <- URL -> page mapping, guards, lazy loading
|
|-- core/                   <- singletons / app-wide logic (no UI pages)
|   |-- auth/               <- AuthService, authGuard, roleGuard, user.model
|   |-- repositories/       <- Inspection repository interface, Local, Remote, Sync,
|   |                          plus InspectionAdapter (DTO <-> domain <-> local entity)
|   |-- services/           <- NetworkService, CameraService, ToastService, mock-data
|   `-- sync/               <- SyncQueueService + sync.model (queue item types)
|
|-- features/               <- one folder per user-facing area (pages)
|   |-- auth/login/         <- LoginPage
|   |-- dashboard/          <- DashboardPage
|   |-- inspections/        <- InspectionListPage, InspectionFormPage, InspectionDetailsPage
|   `-- approvals/          <- ApprovalListPage (supervisor only)
|
`-- shared/                 <- reusable pieces used by several features
    |-- components/         <- SyncBadge, SignaturePad, QrScannerModal, ImagePreviewModal
    |-- models/             <- Inspection, InspectionPhoto, AuditEntry, AssetReference...
    `-- pipes/              <- StatusColorPipe
```

KEY IDEA

Rule of thumb: `core/` = things that exist **once** for the whole app and have no screen. `features/` = things a user can **navigate to**. `shared/` = small building blocks **reused** by multiple features. This convention keeps big Angular apps tidy.

2.3 The configuration files explained

File	Purpose (and important lines)
`package.json`	Lists libraries. **Runtime:** @angular/* 22.1.7, @ionic/angular 9, @capacitor/core 8.5.2, camera, network, preferences, app, haptics, keyboard, status-bar, @ionic/pwa-elements, ionicons, rxjs. **Dev:** @angular/cli & build, typescript ~6, vitest, eslint, pdfkit. Scripts: `start` = `ng serve`, `build` = `ng build`, `test` = `ng test`, `lint` = `ng lint`.
`angular.json`	Defines project `app`. `build` uses builder `@angular/build:application` with entry `src/main.ts`, index `src/index.html`, output folder `www`, styles `global.scss` + `variables.scss`, assets from `src/assets`. `polyfills: []` (no zone.js). `production` config adds size budgets and swaps `environment.ts` with `environment.prod.ts`. `serve` uses the dev-server builder, default configuration `development`. `test` uses `@angular/build:unit-test` with `src/test-setup.ts`.
`capacitor.config.ts`	`appId: io.ionic.fieldinspector`, `appName: Field Inspector & Asset Maintenance`, `webDir: www`. Capacitor copies whatever is inside `www/` into the native project.
`ionic.config.json`	Tells the Ionic CLI this is an `angular-standalone` project with the Capacitor integration enabled.
`tsconfig.json`	`strict: true` (safer types), `target: es2022`, `moduleResolution: bundler`, Angular `strictTemplates: true` (templates are type-checked), `useDefineForClassFields: false` (needed by Angular decorators).
`eslint.config.js`	Lint rules (angular-eslint + typescript-eslint) run by `npm run lint`.
`environments/*`	`environment.ts` -> `production: false`; `environment.prod.ts` is substituted at production build time.
`www/`	Generated by `ng build`. It is the folder Capacitor reads (`webDir`). You never edit it by hand.

> **NOTE**
> The folder also contains helper files unrelated to the app itself: `build_comprehensive_guide.py`, `generate_documentation_pdf.js`, `FieldGuard_Pro_Codebase_Architecture_Guide.pdf`, `dummy.pdf`, `test_run.pdf`. They are earlier documentation experiments - the running app does not use them.

3. Where the Code Begins

The complete journey from typing `npm start` to seeing the first screen.

This is the single most important chapter for understanding the codebase. Follow the numbered steps and open each file mentioned - you will see exactly how the application wakes up.

3.1 The 30-second answer

ANSWER

Entry point = `src/main.ts`, loaded by `src/index.html`. `main.ts` calls `bootstrapApplication(AppComponent, ...)`. `AppComponent` (selector `app-root`) renders `<ion-app><ion-router-outlet/></ion-app>`. The **router** then reads the URL and decides which page to show (`app.routes.ts`).

3.2 Step-by-step startup sequence



Figure 3.1 - From `npm start` to the first rendered screen.

3.3 Reading main.ts line by line

src/main.ts (19 lines - the whole bootstrap)

```
import { bootstrapApplication } from '@angular/platform-browser';
import { RouteReuseStrategy, provideRouter, withComponentInputBinding,
         withPreloading, PreloadAllModules } from '@angular/router';
import { IonicRouteStrategy, provideIonicAngular } from '@ionic/angular';
import { defineCustomElements } from '@ionic/pwa-elements/loader';

import { routes } from './app/app.routes';
import { AppComponent } from './app/app.component';

// Register PWA elements for web camera support
defineCustomElements(window);

bootstrapApplication(AppComponent, {
  providers: [
    { provide: RouteReuseStrategy, useClass: IonicRouteStrategy },
    provideIonicAngular(),
    provideRouter(routes, withPreloading(PreloadAllModules), withComponentInputBinding()),
  ],
});
```

Line / piece	Meaning
`defineCustomElements(window)`	Registers *web components* from `@ionic/pwa-elements` (e.g. `<pwa-camera-modal>`). When the app runs in a browser (no native camera), Capacitor Camera shows this UI instead. Without this call, taking photos on desktop would fail.
`bootstrapApplication(AppComponent, ...)`	The **standalone** way to start Angular - no `AppModule`/`NgModule` needed. Angular creates `AppComponent` and attaches it to `<app-root>`.
`RouteReuseStrategy -> IonicRouteStrategy`	Replaces Angular's default so Ionic can keep previous pages alive in a **navigation stack** (like a native app). This is why Ionic pages have lifecycle hooks such as `ionViewWillEnter`.
`provideIonicAngular()`	Registers Ionic's services (ModalController, ToastController, AlertController, ActionSheetController, Platform...) and global configuration.
`provideRouter(routes, ...)`	Installs the router with our `routes` array. `withPreloading(PreloadAllModules)` downloads lazy chunks after start-up. `withComponentInputBinding()` lets route params be bound to component `@Input()`s (enabled, available for use).

3.4 The root component

The root component is almost empty on purpose

```
// app.component.ts
@Component({
  selector: 'app-root',          // matches <app-root> in index.html
  standalone: true,
  templateUrl: 'app.component.html',
  imports: [IonApp, IonRouterOutlet],   // only the components its template uses
})
export class AppComponent { constructor() {} }

<!-- app.component.html -->
<ion-app>
  <ion-router-outlet></ion-router-outlet>
</ion-app>
```

- `<ion-app>` is Ionic's root container: it handles safe areas, theming, modals/overlays and hardware back-button behaviour.
- `<ion-router-outlet>` is a **placeholder** where the router injects the *current page*. It is Ionic's version of Angular's `<router-outlet>` and adds animated page transitions and the page stack.
- There is **no logic** in `AppComponent`. All behaviour is delegated to pages (features) and services (core).

3.5 The routing table (app.routes.ts)

URL path	Page loaded	Guards	Notes
`''` (empty)	redirect -> `dashboard`	-	`pathMatch: 'full'` = only the exact empty URL
`login`	LoginPage	-	public page
`dashboard`	DashboardPage	authGuard	home after login
`inspections`	InspectionListPage	authGuard	supports `?filter=` query param
`inspections/new`	InspectionFormPage	authGuard	create mode
`inspections/:id`	InspectionDetailsPage	authGuard	`:id` = route parameter
`inspections/:id/edit`	InspectionFormPage	authGuard	same page, edit mode
`approvals`	ApprovalListPage	authGuard + roleGuard	`data: { role: 'SUPERVISOR' }`
`**`	redirect -> `dashboard`	-	wildcard = unknown URLs

> **WATCH OUT**
> Order matters: Angular picks the **first** matching route. `inspections/new` is declared *before* `inspections/:id`, otherwise the word "new" would be captured as an id.

Anatomy of one route

```
{
  path: 'dashboard',
  loadComponent: () => import('./features/dashboard/dashboard.page')
                       .then((m) => m.DashboardPage),   // LAZY LOADING
  canActivate: [authGuard],                             // GUARD
},
```

- **Lazy loading:** `import(...)` is a *dynamic import*. The browser downloads that page's code **only when needed**, keeping the first load small (that is why `www/` contains many `chunk-*.js` files).
- **Guard:** `canActivate` runs *before* navigation. If it returns `true` the page opens; if it returns a `UrlTree` the router redirects there instead.

3.6 What really wakes up when the Dashboard opens

Angular creates objects **lazily** - a service is only constructed the first time something asks for it through `inject(...)`. Opening the dashboard therefore triggers a *cascade* of constructions. Understanding this cascade explains where seeding, listeners and background sync begin:

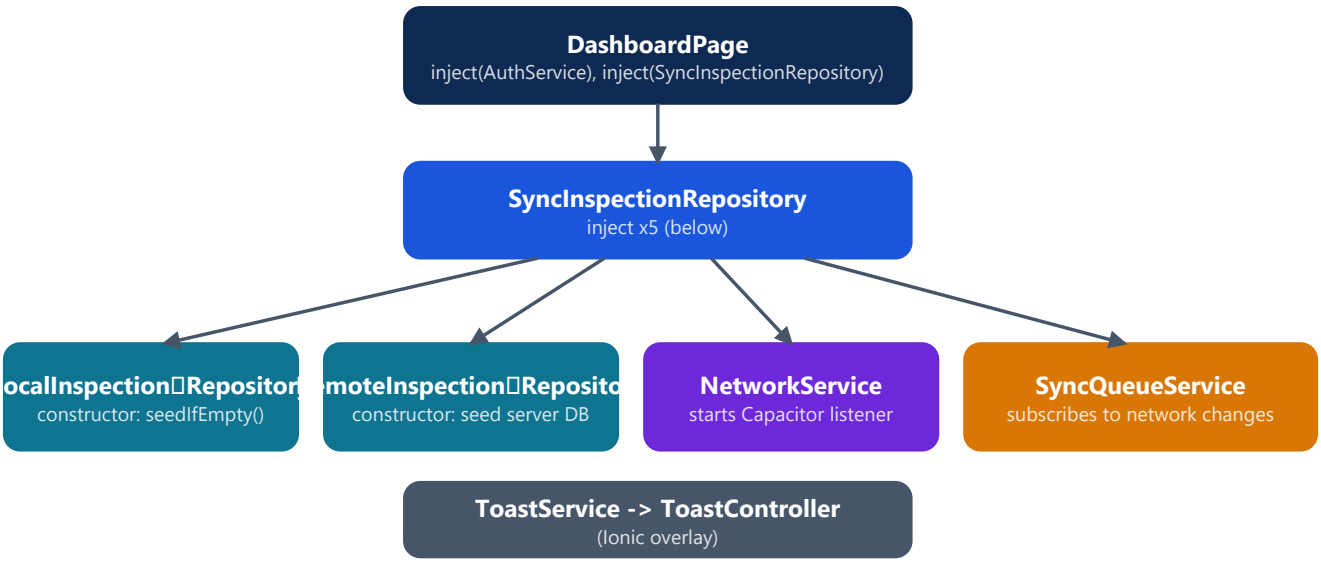


Figure 3.2 - Dependency cascade created by the first `inject(SyncInspectionRepository)`.

- `LocalInspectionRepository` seeds **browser storage** with demo inspections if none exist yet.
- `RemoteInspectionRepository` seeds a **fake server database** (also in browser storage) with all non-draft demo inspections.
- `NetworkService` asks Capacitor for the current connectivity and **starts listening** for changes.
- `SyncQueueService` loads any previously queued offline work and subscribes to network changes so it can **auto-sync** the moment you are back online.

4. TypeScript & Angular Concepts

The language features and framework ideas used throughout the code.

4.1 TypeScript concepts you will meet

Concept	Example from this codebase	What it means
Type alias / union	`type InspectionStatus = 'DRAFT' | 'PENDING_APPROVAL' | 'APPROVED' | 'REJECTED'`	A variable can only hold one of these exact strings. Typos become compile errors.
Interface	`interface Inspection { id: string; assetId: string; ... }`	A *shape* contract for objects; erased at runtime.
Optional property	`damageDescription?: string`	May be `undefined`.
Generics	`signal<Inspection[]>([])`, `Observable<Inspection | null>`	Reusable types parameterised by another type.
Async / await	`async capturePhoto(): Promise<InspectionPhoto | null>`	Write asynchronous code that reads top-to-bottom.
Static members	`InspectionAdapter.toDomainFromApi(dto)`	A function on the class itself, no `new` needed.
Optional chaining / nullish	`session?.user ?? null`	Safe access; fallback when null/undefined.
Spread operator	`{ ...inspection, status: 'DRAFT' }`	Copy an object and override fields (immutability).
Definite assignment `!`	`@ViewChild('sigCanvas') canvasRef!: ElementRef`	Promise to the compiler it will be set later.
Dynamic import	`import('./login.page').then(m => m.LoginPage)`	Load code on demand (lazy loading).

4.2 Standalone components

Before Angular 14, every component had to be declared in an `NgModule`. A **standalone component** lists what it needs **itself** in its `imports` array - no module. Every page and shared component here is `standalone: true`.

Typical component header

```
@Component({
  selector: 'app-sync-badge',          // how you write it in HTML: <app-sync-badge>
  standalone: true,
  imports: [CommonModule, IonBadge, IonButton, IonIcon, IonSpinner],  // dependencies
  template: `...inline HTML...`,       // inline template (these pages use inline ones)
  styles: [`...scoped CSS...`],        // styles apply ONLY to this component
})
export class SyncBadgeComponent { ... }
```

IN PLAIN ENGLISH

A component = **TypeScript class** (data & behaviour) + **template** (HTML) + **styles** (CSS), tied together by the `@Component` decorator. Angular encapsulates the CSS so styles written in one page do not leak into another.

4.3 Templates: bindings and the new control flow

Syntax	Name	Example in this project
`{{ expr }}`	Interpolation	`{{ totalCount() }}` prints a value
`[prop]="expr"`	Property binding	`[disabled]="loginForm.invalid"`, `[src]="photo.dataUrl"`
`(event)="fn()"`	Event binding	`(click)="quickLogin('EMPLOYEE')"`, `(ionChange)="onDamageToggle()"`
`[(ngModel)]`	Two-way binding	`<ion-searchbar [(ngModel)]="searchQuery">`
`[formGroup]`, `formControlName`	Reactive-form binding	Login & inspection forms
`@if (...) { } @else { }`	Control flow (Angular 17+)	Show online/offline chip; conditional damage fields
`@for (x of list; track x.id)`	Control flow loop	Photo grid, inspection list; `track` helps Angular reuse DOM nodes
`value | statusColor:'sync'`	Pipe	Converts `PENDING_SYNC` into the Ionic colour `warning`

4.4 Dependency Injection (DI)

Instead of a class creating its own helpers (`new AuthService()`), it **asks** Angular for them. Angular builds the object once, shares it, and hands it over. This project uses the modern function style:

inject() + providedIn: 'root'

```
@Injectable({ providedIn: 'root' })     // ONE shared instance for the whole app
export class AuthService {
  private router = inject(Router);        // Angular gives us the Router
  private toast  = inject(ToastService);  // ...and our own ToastService
}
```

- **Singleton:** `providedIn: 'root'` means every class that injects `AuthService` receives the **same** object - so login state is shared everywhere.
- **Testability:** in unit tests you swap a real service for a fake: `{ provide: NetworkService, useValue: mockNetwork }` (see `sync-inspection.repository.spec.ts`).
- **Constructor-free:** `inject()` works in field initialisers, guards (plain functions) and constructors.

4.5 Signals - the reactive state primitive

A **signal** is a box holding a value that **notifies** whoever reads it when it changes. Angular 22 uses signals to know exactly which parts of the screen need updating.

API	Role	Example
`signal(initial)`	Writable state	`private sessionSignal = signal<AuthSession | null>(...)`
`.set(v)` / `.update(fn)`	Change it	`this.photos.update(arr => [...arr, photo])`
`signal()` read call	Read value (note the parentheses!)	`totalCount()`, `network.isOnline()`
`computed(() => ...)`	Derived read-only value, auto-recalculated & cached	`isAuthenticated`, `approvedCount`, `filteredInspections`

API	Role	Example
`.asReadonly()`	Expose without letting others write	`currentSession = sessionSignal.asReadonly()`

Signal + computed in practice

```
// DashboardPage - a derived number that updates by itself
private inspectionsSignal = signal<Inspection[]>([]);
public readonly approvedCount = computed(
  () => this.inspectionsSignal().filter(i => i.status === 'APPROVED').length
);
// template:  {{ approvedCount() }}      <- re-renders when inspectionsSignal changes
```

IMPORTANT SUBTLETY

`computed()` only re-runs when a **signal it read** changes. If a `computed` reads a plain class field (not a signal) that field is invisible to Angular's tracking. Two pages in this repo do exactly that - see Chapter 15 ("Honest notes") for a concrete example worth fixing.

No zone.js: `angular.json` has `polyfills: []`. The app does not load zone.js, so it relies on signals and template event handlers to trigger screen updates (zoneless change detection).

4.6 RxJS Observables

An **Observable** is a stream of values over time (like a promise that can emit many values). Repositories return `Observable<...>` to look like real HTTP calls so a real backend can be plugged in later.

Operator / function	Use in this code
`of(value)`	Wrap an immediate value as an Observable (local reads).
`delay(ms)`	Simulate server latency in `RemoteInspectionRepository` (150-300 ms).
`throwError(() => err)`	Emit an error - used when offline: `Network Error: Device is offline`.
`switchMap`	After one Observable emits, continue with another (`localRepo.save` -> `remoteRepo.post`).
`map`	Transform the result (show a toast then return the saved object).
`catchError`	Recover from failures - fall back to offline queue.
`firstValueFrom`	Convert an Observable to a Promise so `await` can be used (sync queue).
`Subject` / `.asObservable()`	Manual event stream: `NetworkService.onStatusChange`.
`.subscribe(cb)`	Actually *run* the Observable. Nothing happens until someone subscribes!

4.7 Component lifecycle hooks

Hook	Who provides it	Used for
`ngOnInit()`	Angular	First data load (`loadData()`), subscribe to route params.
`ngAfterViewInit()`	Angular	SignaturePad: the `<canvas>` exists now, so attach drawing listeners.
`ionViewWillEnter()`	**Ionic**	Runs **every time** a cached page becomes visible again - refreshes lists after you return from another screen.

KEY IDEA

Because `IonicRouteStrategy` **keeps pages alive** in the stack, `ngOnInit` runs only **once** per page instance. That is why the dashboard, list and approvals pages also implement `ionViewWillEnter` to re-read data when you come back.

4.8 Reactive Forms and conditional validation

The login and inspection pages use **Reactive Forms**: the form's structure and rules live in TypeScript.

Conditional validation (inspection-form.page.ts)

```
inspectionForm: FormGroup = this.fb.group({
  assetId:        ['', [Validators.required]],
  inspectionDate: [new Date().toISOString().substring(0, 16), [Validators.required]],
  conditionStatus:['GOOD', [Validators.required]],
  hasDamage:      [false, [Validators.required]],
  damageDescription: [''],            // required ONLY when hasDamage is true
  severity:       ['LOW'],            // required ONLY when hasDamage is true
  notes:          ['', [Validators.required]],
});

onDamageToggle() {                     // called on radio change
  const hasDamage = this.inspectionForm.get('hasDamage')?.value;
  if (hasDamage) { desc.setValidators([Validators.required]); sev.setValidators([Validators.required]); }
  else           { desc.clearValidators();                    sev.clearValidators(); }
  desc.updateValueAndValidity();  sev.updateValueAndValidity();   // re-run the rules
}
```

- `FormBuilder.group({...})` builds a `FormGroup` of `FormControl`s (initial value + validators).
- In the template `[formGroup]="inspectionForm"` and `formControlName="assetId"` connect inputs to controls.
- `markAllAsTouched()` before submit makes every error message appear even if the user never visited the field.

4.9 Guards, pipes, @Input/@Output, @ViewChild

Feature	Where	Explanation
Functional guard `CanActivateFn`	`auth.guard.ts`, `role.guard.ts`	A plain function using `inject()`. Returns `true` or a `UrlTree` redirect. `roleGuard` reads `route.data['role']`.
Pure pipe	`StatusColorPipe`	Transforms a value in the template; reusable & pure (same input -> same output).
`@Input()`	`SignaturePad.title`, `.existingSignature`	Parent passes data **down**: `[existingSignature]="existingSignature"`.
`@Output()` + `EventEmitter`	`SignaturePad.signatureChange`	Child sends events **up**: `(signatureChange)="onSignatureChange($event)"`.
`@ViewChild('sigCanvas')`	`SignaturePad`	Grab a reference to the `<canvas #sigCanvas>` element to draw on it.

5. Ionic Framework Concepts

The UI toolkit that makes a web app look and behave like a native mobile app.

5.1 What is Ionic?

Ionic is a library of **ready-made mobile UI components** (buttons, lists, cards, modals, tabs, toasts, action sheets...) plus a **navigation system** that mimics native page stacks. The components are built as standard **web components** (custom HTML tags prefixed with `ion-`). `@ionic/angular` wraps each one so you can import it into an Angular standalone component.

Ionic components are imported one by one (tree-shakable)

```
import { IonButton, IonContent, IonHeader, IonToolbar, IonTitle } from '@ionic/angular';

@Component({
  standalone: true,
  imports: [IonButton, IonContent, IonHeader, IonToolbar, IonTitle],  // import ONLY what you use
  template: `<ion-header><ion-toolbar><ion-title>Hi</ion-title></ion-toolbar></ion-header>
             <ion-content><ion-button>Tap</ion-button></ion-content>`,
})
```

> **IN PLAIN ENGLISH**
>
> Ionic **automatically adapts its look** to the platform: Material Design on Android ("md" mode) and iOS styling on iPhones ("ios" mode). You write the markup once.

5.2 Anatomy of an Ionic page

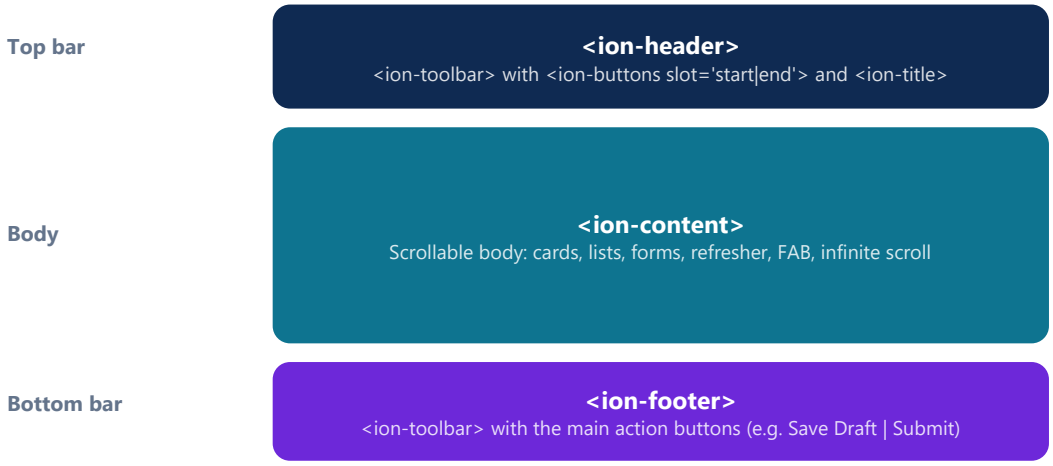


Figure 5.1 - Every page in the app follows this skeleton (see InspectionFormPage).

5.3 Ionic components used in this project

Component	What it does	Where used
`ion-app`, `ion-router-outlet`	Root container & page host	`app.component.html`
`ion-header/toolbar/title/buttons`	Top navigation bar	All pages

Component	What it does	Where used
`ion-back-button defaultHref`	Smart back arrow; `defaultHref` is used when there is no history (e.g. deep link)	List, Form, Details
`ion-content`	Scrollable area with pull-to-refresh/FAB support	All pages
`ion-card*`	Card container (header, title, content)	Login, form sections, details
`ion-list`, `ion-item`, `ion-label`	Row-based lists; `button` makes an item tappable	Inspection lists, QR asset list
`ion-input`, `ion-textarea`, `ion-select`, `ion-radio-group`	Form controls (integrate with Angular forms)	Login, inspection form
`ion-button`, `ion-icon`, `ion-badge`, `ion-chip`, `ion-note`, `ion-spinner`	Actions, icons, status pills, hints, loading	Everywhere
`ion-segment` + `ion-segment-button`	Tab-like filter switch	Inspection list filters; Approvals Pending/Resolved
`ion-searchbar`	Search box with `debounce`	Inspection list
`ion-refresher`	Pull-to-refresh (`ionRefresh` event, call `event.target.complete()`)	Dashboard, lists
`ion-infinite-scroll`	Load more on scroll (`ionInfinite`)	Inspection list (PAGE_SIZE paging)
`ion-fab` + `ion-fab-button`	Floating action button	New inspection from list
`ion-grid/row/col`	12-column responsive grid (`size`, `size-md`)	Dashboard metrics, photo grid

5.4 Slots, colours and properties

- **Slots** place content inside a component: `<ion-icon slot="start">` puts an icon before the button text; `slot="icon-only"` makes a square icon button; `<ion-refresher slot="fixed">` pins it outside scroll content.
- **Colour names** such as `color="primary"`, `"success"`, `"warning"`, `"danger"`, `"medium"` map to CSS variables defined in `theme/variables.scss` (e.g. `--ion-color-primary: #1a56db`).
- **fill / expand / size:** `fill="outline" | "clear" | "solid"`, `expand="block"` (full width), `size="small"`.
- **Component CSS variables:** you can restyle internals without hacking: `--background`, `--border-radius`, `--padding-start`.

5.5 Navigation: how Ionic differs from plain Angular

In a normal website, navigating away destroys the old page. In a mobile app you expect to **go back** and find the previous screen exactly as you left it. Ionic achieves this by keeping pages in a **stack**:

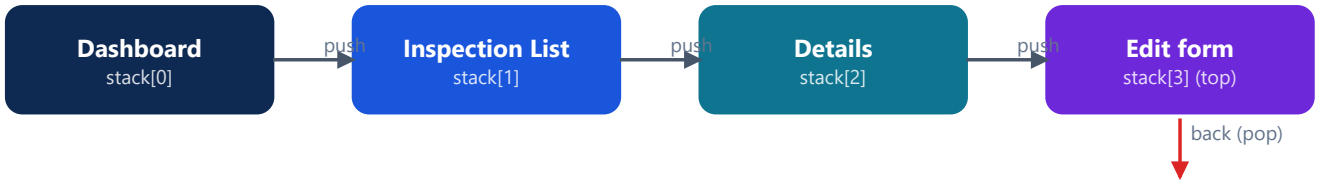


Figure 5.2 - Pages below the top stay alive in memory (hidden), so back navigation is instant.

- `IonicRouteStrategy` (set in `main.ts`) is what enables this.
- `routerLink="['/inspections', item.id]"` pushes a page; `<ion-back-button>` pops it.
- Hidden pages are **not destroyed**, so their `ngOnInit` does not run again. Ionic fires `ionViewWillEnter` / `ionViewDidEnter` / `ionViewWillLeave` instead - this app uses `ionViewWillEnter` to refresh data.
- Page transitions are animated automatically and follow platform conventions.

5.6 Overlay controllers (programmatic UI)

Toasts, alerts, action sheets and modals are not placed in HTML. You **inject a controller**, create the overlay in code and present it.

Controller	Used for	File
`ToastController`	Short messages (success/error/warning/info) at top or bottom	`ToastService` wraps 4 helpers: `showSuccess/Error/Warning/Info`
`AlertController`	Confirm dialogs with inputs (approve notes, mandatory reject reason)	Approvals + Details pages
`ActionSheetController`	Bottom menu: *Sync Queue Now*, *Simulate Offline*	`SyncBadgeComponent`
`ModalController`	Full-screen dialogs hosting a component	QR scanner, photo preview

Modal round-trip (inspection-form -> image preview)

```
// 1. Open a modal showing a component, pass data in, wait for a result
const modal = await this.modalCtrl.create({
  component: ImagePreviewModalComponent,
  componentProps: { photo, canDelete: true },       // becomes @Input()s of the modal
});
await modal.present();
const { data, role } = await modal.onWillDismiss();   // resolves when it closes
if (role === 'delete' && data?.photoId) { /* remove the photo */ }

// 2. Inside the modal component, close it and return a result
this.modalCtrl.dismiss({ photoId: this.photo.id }, 'delete');   // (data, role)
```

TIP

`role` is a small string label (`'selected'`, `'delete'`, `'cancel'`) the opener uses to decide what to do. It is a neat way to return *intent* from a modal.

5.7 Icons

Icons come from the **ionicons** package. They are **registered** before use, then referenced by kebab-case name:

Registering icons keeps the bundle small (only icons you import ship)

```
import { addIcons } from 'ionicons';
import { cameraOutline, trashOutline } from 'ionicons/icons';
constructor() { addIcons({ cameraOutline, trashOutline }); }   // register
// template:   <ion-icon name="camera-outline"></ion-icon>      // use
```

5.8 Theming and global CSS

File	What it does
`global.scss`	Imports Ionic's required CSS (`core`, `normalize`, `structure`, `typography`), optional utility classes (`padding`, `flex-utils`, `text-alignment`...) and `palettes/dark.system.css` -> dark mode follows the device setting.
`theme/variables.scss`	Defines the brand palette: primary `#1a56db`, secondary `#0e7490`, tertiary `#475569`, success `#16a34a`, warning `#d97706`, danger `#dc2626`, medium `#64748b`, light, dark. Each colour has `-rgb`, `-contrast`, `-shade`, `-tint` variants Ionic needs.
Component `styles: [...]`	Per-page SCSS written inside the `@Component` decorator (encapsulated).

5.9 @ionic/pwa-elements

When Capacitor's Camera plugin runs in a **browser** there is no native camera screen. `@ionic/pwa-elements` supplies web components (camera capture modal) so the same `Camera.getPhoto()` code still works on desktop. It is registered once in `main.ts` with `defineCustomElements(window)` and is unnecessary inside a native app.

6. Capacitor Concepts

How a web app becomes a real Android/iOS app and talks to device features.

6.1 What is Capacitor?

Capacitor is a **native runtime**. It creates a native Android/iOS project containing a **WebView** that loads your built web app (`www/`). A **bridge** lets your JavaScript call native code through **plugins** and get results back as Promises.

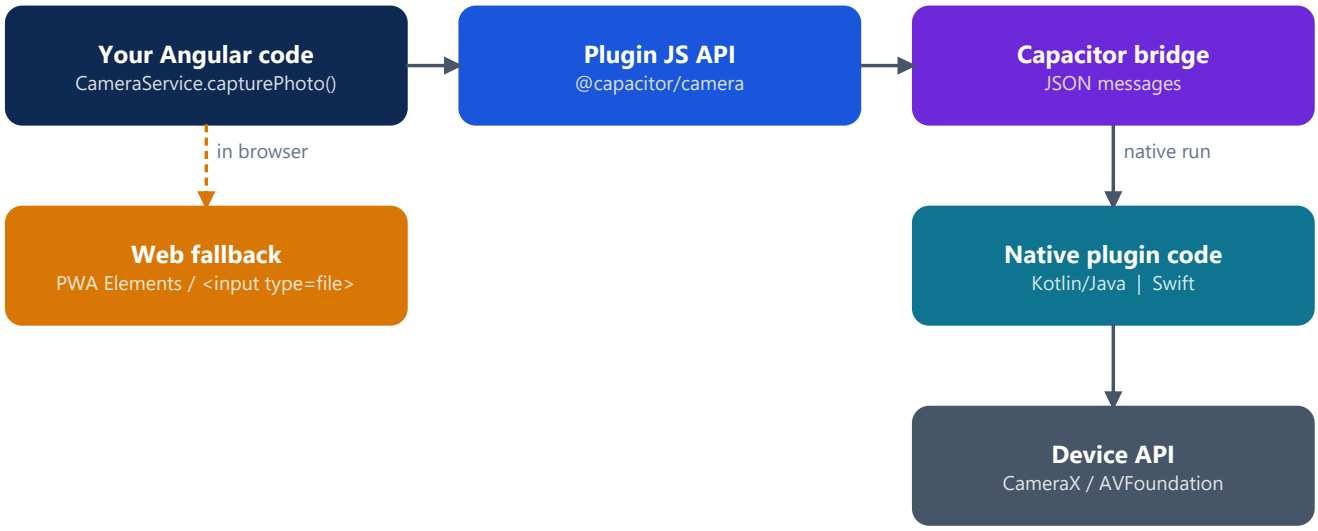


Figure 6.1 - Same TypeScript call; Capacitor routes it to the native layer on a phone or to a web fallback in the browser.

6.2 capacitor.config.ts

Only three settings are customised - the minimum

```
import type { CapacitorConfig } from '@capacitor/cli';

const config: CapacitorConfig = {
  appId: 'io.ionic.fieldinspector',              // reverse-DNS app identifier (store id)
  appName: 'Field Inspector & Asset Maintenance', // name shown under the home-screen icon
  webDir: 'www'                                  // folder with the built web app
};
export default config;
```

KEY IDEA

`webDir: 'www'` matches `angular.json -> outputPath.base: 'www'`. That link is what lets `npx cap sync` copy the Angular build into the native projects.

6.3 Plugins: installed vs. actually used

Package	Purpose	Status in code
`@capacitor/core`	Runtime, `registerPlugin`, platform detection	Dependency of all plugins
`@capacitor/camera`	Take photos / pick from gallery, permissions	**USED** - `CameraService`

Package	Purpose	Status in code
`@capacitor/network`	Current connectivity + change events	**USED** - `NetworkService`
`@capacitor/preferences`	Native key-value storage (survives better than localStorage)	Installed, **not imported yet**
`@capacitor/app`	App lifecycle, back button, deep links	Installed, not used yet
`@capacitor/haptics`	Vibration feedback	Installed, not used yet
`@capacitor/keyboard`	Keyboard behaviour on mobile	Installed, not used yet
`@capacitor/status-bar`	Status bar colour/style	Installed, not used yet
`@capacitor/cli`	`npx cap ...` commands (dev dependency)	Tooling

6.4 The Camera plugin in this app

core/services/camera.service.ts (condensed)

```
async capturePhoto(): Promise<InspectionPhoto | null> {
  const ok = await this.checkOrRequestPermissions();        // 1. permissions
  if (!ok) { toast('Camera permission was denied...'); return null; }
  try {
    const photo = await Camera.getPhoto({                   // 2. open camera
      quality: 85, allowEditing: false,
      resultType: CameraResultType.DataUrl,                 //    -> base64 'data:image/jpeg;...'
      source: CameraSource.Camera,  width: 1280,            //    downscale to 1280px wide
    });
    const sizeKb = Math.round((photo.dataUrl.length * 3) / 4 / 1024);   // 3. estimate size
    return { id: 'photo_' + Date.now() + '_' + rand, dataUrl: photo.dataUrl,
             timestamp: new Date().toISOString(), sizeKb };
  } catch (err) {
    if (userCancelled(err)) return null;                    // quiet exit
    return this.pickPhotoFromFilePicker();                  // 4. web fallback
  }
}
```

- **Permissions:** `Camera.checkPermissions()` then `requestPermissions()`. In a browser these calls may throw, so the `catch` simply **allows continuing**.
- `CameraSource.Camera` **vs** `CameraSource.Photos`**:** camera vs gallery (`selectFromGallery()`).
- `CameraResultType.DataUrl`**:** the image arrives as a base64 string, which is easy to store inside the inspection JSON and show with `<img [src]>`.
- **Size formula:** base64 text is ~4/3 the size of the binary, hence `length * 3 / 4`.
- **Fallback chain:** native camera -> PWA camera modal (browser) -> hidden `<input type=file accept=image/*>` created in code (`pickPhotoFromFilePicker`).

6.5 The Network plugin in this app

core/services/network.service.ts (condensed)

```
const status = await Network.getStatus();            // { connected: boolean, connectionType }
this.rawOnlineSignal.set(status.connected);

Network.addListener('networkStatusChange', (status) => {   // fires on connect/disconnect
  this.rawOnlineSignal.set(status.connected);
  this.statusChanges$.next(this.isOnline());               // notify RxJS subscribers
});

// "effective" status = real network AND not in demo-simulated offline mode
public readonly isOnline = computed(() => this.rawOnlineSignal() && !this.simulatedOfflineSignal());
```

TIP

The **Simulate Offline Mode** toggle (in the Sync Badge action sheet) flips `simulatedOfflineSignal`. This lets you demo and test offline behaviour on a desktop browser without touching your Wi-Fi.

6.6 Build and run on a real device

Typical Capacitor workflow

```
# one-time: add native platforms (their npm packages are not in package.json yet)
npm install @capacitor/android @capacitor/ios
npx cap add android            # creates android/ project
npx cap add ios                # creates ios/ project (macOS + Xcode required)

# every time you change web code
npm run build                  # Angular -> www/
npx cap sync                   # copy www/ + update native plugins

npx cap open android           # open in Android Studio, press Run
npx cap run android -l --external   # live-reload on a connected device
```

WATCH OUT

On **iOS** you must add usage-description strings (e.g. camera and photo-library reasons) to `Info.plist`, otherwise the OS will block access. On **Android** the plugin declares the permissions it needs when you sync. Always read the plugin's docs when you add a native platform.

6.7 Storage on a device

This POC stores data with the browser's `localStorage` (inside the WebView). That is fine for a demo, but the OS may clear WebView storage under memory pressure and `localStorage` has a small quota (~5 MB) while photos are stored as base64. Production apps normally move to `@capacitor/preferences` for small settings and **SQLite / filesystem** for inspections and photos. Because all data access is hidden behind repositories (Chapter 7), that migration only touches `LocalInspectionRepository`.

7. Architecture & Design Patterns

Why the code is split into repositories, adapters and a sync queue.

7.1 Layered view

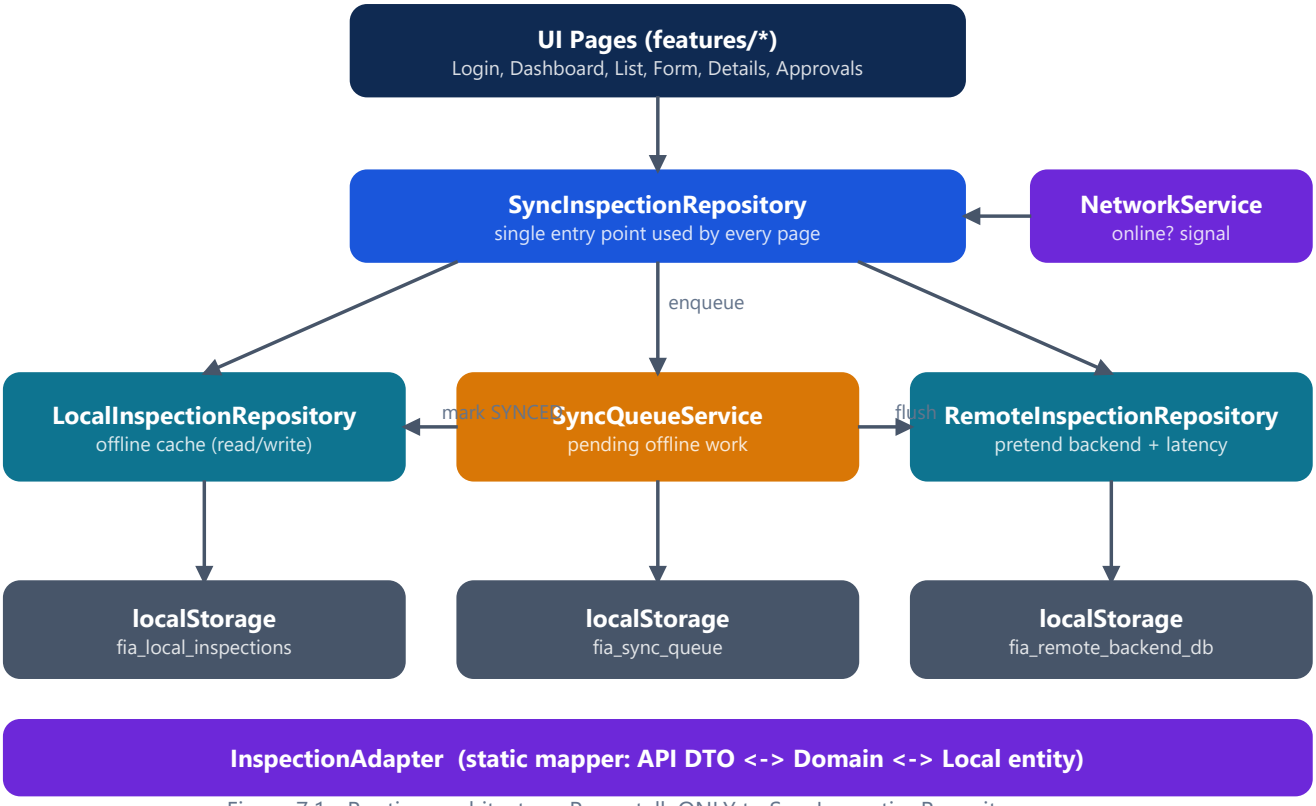


Figure 7.1 - Runtime architecture. Pages talk ONLY to SyncInspectionRepository.

7.2 Repository pattern

A **repository** hides *where data lives* behind a clean set of methods. The page asks "give me all inspections" and does not care whether they come from a phone database, a REST API or memory.

repositories/inspection.repository.ts - the contract

```
export interface InspectionRepository {
  getAll(): Observable<Inspection[]>;
  getById(id: string): Observable<Inspection | null>;
  saveDraft(inspection: Inspection): Observable<Inspection>;
  submit(inspection: Inspection): Observable<Inspection>;
  approve(id, supervisorId, supervisorName, comments?): Observable<Inspection>;
  reject(id, supervisorId, supervisorName, comments): Observable<Inspection>;
  delete(id: string): Observable<boolean>;
}
```

Class	Role	Key methods
`SyncInspectionRepository`	**Implements the contract.** The orchestrator: decides local vs remote vs queue.	getAll, getById, saveDraft, submit, approve, reject, delete
`LocalInspectionRepository`	Offline cache in `localStorage` (key `fia_local_inspections`); seeds demo data on first run.	getAll, getById, save, bulkUpsert, delete

Class	Role	Key methods
`RemoteInspectionRepository`	**Fake backend** in `localStorage` (key `fia_remote_backend_db`). Refuses to work when offline (throws a network error) and adds artificial delay.	fetchAll, fetchById, postInspection, deleteInspection

> **NOTE**
> Only `SyncInspectionRepository` formally `implements InspectionRepository`. Local and Remote have their own method names (`save`, `fetchAll`...) because they are *lower-level* building blocks. To use a real server later, you would replace the internals of `RemoteInspectionRepository` with `HttpClient` calls - no page changes needed.

7.3 Adapter pattern

Three different **shapes** of the same data exist. `InspectionAdapter` converts between them so that each layer can stay clean:

Shape	Type	Style	Used by
Domain (clean)	`Inspection`	camelCase, rich types	Pages, repositories, queue
Server DTO	`InspectionApiDto`	snake_case (`asset_code`, `workflow_state`, `created_iso`)	RemoteInspectionRepository
Local entity	`InspectionLocalEntity`	`{ id, data: JSON string, syncStatus, updatedAt }`	LocalInspectionRepository

Domain field	API DTO field	Mapping note
`assetId`	`asset_code`	rename
`assetName`	`asset_title`	rename
`location`	`facility_location`	rename
`conditionStatus`	`overall_condition`	string cast to `ConditionStatus`
`hasDamage` / `damageDescription` / `severity`	`damage_detected` / `damage_details` / `severity_rating`	boolean coerced with `!!`
`photos[]` {id, dataUrl, caption, timestamp}	`attached_images[]` {photo_id, image_base64, caption, captured_at}	array `.map`
`gpsLocation {latitude, longitude}`	`geo_coordinates {lat, lng}`	nested rename
`status`	`workflow_state`	workflow state
`inspectorId/Name`, `supervisorId/Name`	`auditor_id/name`, `reviewer_id/name`	different vocabulary
`history[]` audit entries	`audit_log[]` {log_id, logged_at, user_*, event_type, comment}	array `.map`

> **KEY IDEA**
> Why bother? If the backend team renames `asset_code` tomorrow, **only the adapter changes**. Every screen continues to use `assetId`. When data comes from the server the adapter also stamps `syncStatus: 'SYNCED'` automatically.

7.4 Offline-first strategy

- **Reads:** always answer instantly from the local cache. If online, refresh from the server in the background and merge (never overwriting items that are still `PENDING_SYNC`).
- **Writes:** save locally first. If online try the server; if the server call fails or the device is offline, put the work on the **sync queue**.
- **Reconnect:** `SyncQueueService` listens to `NetworkService.onStatusChange` and automatically flushes the queue.
- **Visibility:** badges show *Draft / Pending Sync / Synced / Sync Failed* so the user always knows the state of their work.

8. Domain Model & State Machines

The data shapes and the lifecycles an inspection moves through.

8.1 The Inspection model

Group	Fields	Notes
Identity	`id`	`'insp_' + Date.now() + random` for new ones
Asset	`assetId`, `assetName`, `assetCategory`, `location`	Auto-filled from `MOCK_ASSETS` when the id matches
Findings	`conditionStatus` (GOOD/FAIR/POOR/CRITICAL), `hasDamage`, `damageDescription?`, `severity?` (LOW/MEDIUM/HIGH/CRITICAL), `notes`	Damage fields mandatory only when `hasDamage` is true
Evidence	`photos[]` (`id`, `dataUrl`, `caption?`, `timestamp`, `sizeKb?`), `signatureUrl?`, `gpsLocation?`	Images and signature are base64 data URLs
Workflow	`status` (DRAFT / PENDING_APPROVAL / APPROVED / REJECTED)	Business process state
Sync	`syncStatus` (DRAFT / PENDING_SYNC / SYNC_FAILED / SYNCED), `syncError?`, `lastSyncedAt?`	Technical delivery state
People	`inspectorId/Name`, `supervisorId/Name?`, `reviewComments?`, `reviewedAt?`	Who did what
Audit	`history[]` of `{ id, timestamp, actorId, actorName, actorRole, action, notes? }`	Actions: CREATED, UPDATED, SAVED_DRAFT, SUBMITTED, APPROVED, REJECTED, SYNCED
Timestamps	`createdAt`, `updatedAt`, `inspectionDate`	ISO strings

KEY IDEA

There are **two independent status axes**. `status` answers *"where is this in the review process?"*; `syncStatus` answers *"has this reached the server yet?"*. An inspection can be `PENDING_APPROVAL` (business) and `PENDING_SYNC` (technical) at the same time - exactly what happens when you submit offline.

8.2 Workflow status machine



Figure 8.1 - Only DRAFT items show the Edit & Complete Draft button in the details page.

8.3 Sync status and queue item status

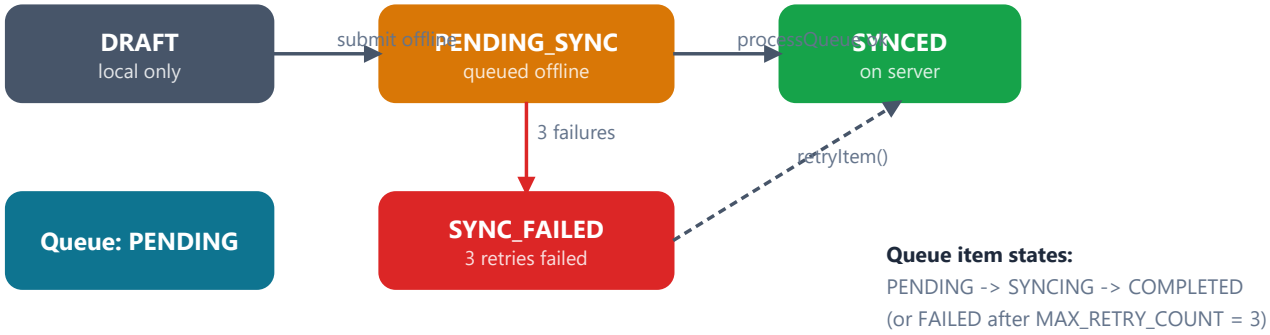


Figure 8.2 - A successful online submit skips straight to SYNCED.

8.4 Roles and permissions

Role	Demo user	Can do	Enforced by
`EMPLOYEE` (Field Engineer)	Alex Carter - `alex@fieldservice.com` (`emp-101`)	Create/save/submit inspections, photos, signature, view lists	`authGuard`; UI hides supervisor actions
`SUPERVISOR`	Sarah Jenkins - `sarah@fieldservice.com` (`sup-201`)	Everything above plus the `/approvals` portal and Approve/Reject buttons	`roleGuard` (`data: {role:'SUPERVISOR'}`), `auth.isSupervisor()` in templates

8.5 Seed data

`core/services/mock-data.ts` provides `MOCK_USERS` (2 users), `MOCK_ASSETS` (6 assets: pump, turbine, compressor, transformer, storage tank, steam manifold) and `INITIAL_INSPECTIONS` (demo inspections with audit history and SVG placeholder photos). They are written to storage the first time the app runs.

9. Core Services Deep Dive

A close reading of each service in `core/`.

9.1 AuthService (core/auth/auth.service.ts)

Member	Behaviour
`sessionSignal`	Private writable signal holding `AuthSession | null` (`token`, `user`, `expiresAt`). Initial value comes from `loadPersistedSession()` - so a page refresh keeps you signed in.
`currentUser`, `userRole`, `isSupervisor`, `isEmployee`	`computed()` values derived from the session.
`isAuthenticated`	`computed`: session exists **and** `expiresAt > Date.now()` (8-hour session).
`login(email, rolePreference?)`	Looks the email up in `MOCK_USERS`; if not found falls back to first user with the preferred role; if still none creates an **ad-hoc EMPLOYEE**. **The password is never verified** (mock auth). Creates a fake token `jwt_mock_xxx` and persists it.
`quickSwitchRole(role)`	Instantly sign in as the demo user for a role (login buttons + dashboard swap icon).
`persistSession(s)`	Writes/removes `localStorage['fia_auth_session']` **and** updates the signal in one place.
`checkSessionValidity()` / `handleSessionExpired()`	If expired: clear session, show warning toast, navigate to `/login`. Called from the constructor.
`logout()`	Clear session, toast, navigate to `/login`.

Guards

auth.guard.ts / role.guard.ts (condensed)

```
export const authGuard: CanActivateFn = () => {
  const auth = inject(AuthService), router = inject(Router);
  return auth.isAuthenticated() ? true : router.createUrlTree(['/login']);
};

export const roleGuard: CanActivateFn = (route) => {
  const required = route.data['role'] as UserRole;       // from the route definition
  if (!auth.isAuthenticated()) return router.createUrlTree(['/login']);
  if (required && auth.userRole() !== required) {
    toast.showWarning(`Access denied: Requires ${required} permissions.`);
    return router.createUrlTree(['/dashboard']);
  }
  return true;
};
```

SECURITY NOTE

Guards protect **navigation**, not data. In a real system the **server** must also verify the token and role on every request - client-side checks alone can be bypassed.

9.2 NetworkService

- Two signals: **raw** device connectivity (from Capacitor / `navigator.onLine`) and **simulated offline** (demo switch).

- `isOnline = raw && !simulated` is the **only** value other code should read.
- `onStatusChange` is an RxJS stream that emits the effective status whenever either input changes.
- Initialisation: `Network.getStatus()` for the starting value; `Network.addListener('networkStatusChange', ...)` for updates, with browser `online`/`offline` events as a fallback.

9.3 ToastService & CameraService

`ToastService` is a thin wrapper over Ionic's `ToastController` with four presets: **success** (green, top), **error** (red, top, longer), **warning** (amber, top), **info** (grey, bottom). Centralising them gives the app one consistent look. `CameraService` was covered in section 6.4.

9.4 SyncInspectionRepository in detail

getAll() - cache first, refresh in background

Instant UI, eventual consistency

```
getAll() {
  return this.localRepo.getAll().pipe(switchMap(localItems => {
    if (this.networkService.isOnline()) {
      this.remoteRepo.fetchAll().pipe(catchError(() => of([]))).subscribe(remoteItems => {
        // merge: replace a local item with the server copy ONLY if the local copy is SYNCED
        // (never overwrite DRAFT / PENDING_SYNC work), add server-only items
        this.localRepo.bulkUpsert(merged).subscribe();
      });
    }
    return of(localItems);          // <- UI gets the instant local answer
  }));
}
```

submit() - three possible paths

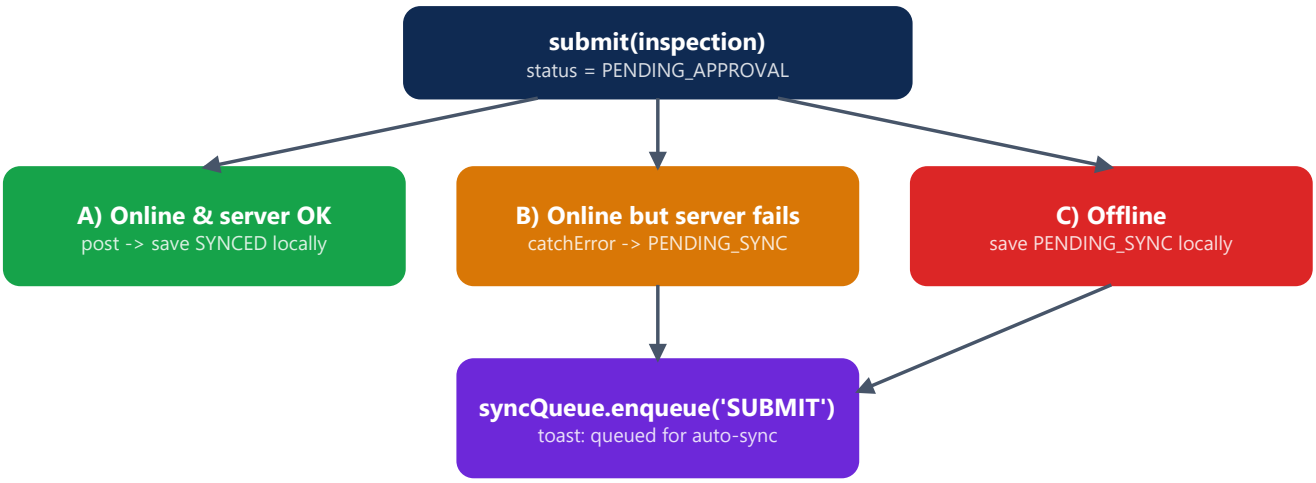


Figure 9.1 - Every path first adds an audit entry (`SUBMITTED`) at the front of `history`.

approve() / reject()

1. `getById(id)` (local first) -> throws `Inspection not found` if missing.
2. Build `updated` with new `status`, supervisor id/name, `reviewComments`, `reviewedAt`, `syncStatus` (SYNCED if online else PENDING_SYNC).
3. Prepend an audit entry (`APPROVED` / `REJECTED`) with actor role `SUPERVISOR`.

4. Online: `remoteRepo.postInspection` then `localRepo.save` and a success/info toast. Offline: save locally, `enqueue('APPROVE' | 'REJECT')` and a warning toast.

saveDraft() and delete()

`saveDraft` forces `status` and `syncStatus` to `DRAFT`, adds a `SAVED_DRAFT` audit entry and writes **only locally** (drafts never leave the device). `delete` removes locally and, if online, also from the server (errors ignored).

9.5 SyncQueueService

core/sync/sync-queue.service.ts (condensed)

```
enqueue(operation, inspection) {
  // one queue item per inspection: re-enqueueing UPDATES the existing item (dedupe)
  const item = { id, inspectionId: inspection.id, operation, payload: inspection,
                 enqueuedAt: now, retryCount: 0, status: 'PENDING' };
  persistQueue(queue);                          // localStorage['fia_sync_queue'] + signals
  if (this.networkService.isOnline()) this.processQueue();
}

async processQueue() {
  if (isSyncing || !online) return;             // re-entrancy + connectivity guard
  for (const item of queue) {
    item.status = 'SYNCING';
    try {
      const saved = await firstValueFrom(remoteRepo.postInspection(item.payload));
      saved.syncStatus = 'SYNCED';  saved.lastSyncedAt = now;
      await firstValueFrom(localRepo.save(saved));      // local copy flips to SYNCED
      item.status = 'COMPLETED';
    } catch (err) {
      item.retryCount++;
      item.status = item.retryCount >= 3 ? 'FAILED' : 'PENDING';
      if (FAILED) { payload.syncStatus = 'SYNC_FAILED'; localRepo.save(payload); }
    }
  }
  queue = queue.filter(q => q.status !== 'COMPLETED');   // drop finished work
  ...toasts: 'Synchronized N inspections' / 'N items failed to sync. Will retry.'
}
```

- **Auto trigger:** the constructor subscribes `networkService.onStatusChange` and calls `processQueue()` when going online with pending items.
- **Manual triggers:** *Sync Queue Now* in the Sync Badge sheet; `retryItem(id)` resets the retry counter and re-processes.
- **Counters:** `pendingCount` and `failedCount` are plain signals refreshed by `updateCounters()` whenever the queue is persisted - the Sync Badge binds to them.
- **Storage:** the full inspection (including base64 photos) is stored inside each queue item.

9.6 All persisted keys

localStorage key	Owner	Content
`fia_auth_session`	AuthService	`{ token, user, expiresAt }`
`fia_local_inspections`	LocalInspectionRepository	Array of `InspectionLocalEntity` (JSON string per inspection)
`fia_remote_backend_db`	RemoteInspectionRepository	Array of `InspectionApiDto` - the "server"
`fia_sync_queue`	SyncQueueService	Array of `SyncQueueItem`

TIP

To **reset the demo to factory state**, open the browser DevTools -> Application -> Local Storage and delete these four keys (or run `localStorage.clear()` in the console), then reload.

10. Feature Pages Walkthrough

Each screen: purpose, dependencies, state and notable techniques.

All pages are **standalone components with inline templates and styles** (single `.ts` file). They follow the same recipe: inject services -> hold state in signals -> load in `ngOnInit` / `ionViewWillEnter` -> render.

10.1 LoginPage (`/login`)

- **Form:** reactive `loginForm` with `email` (required + email format) and `password` (required, min length 4). Pre-filled with `alex@fieldservice.com` / `password123` for demo convenience.
- `onSubmit()`**:** `auth.login(email)` -> success toast -> navigate `/dashboard`.
- `quickLogin(role)`**:** one-tap demo sign-in using `auth.quickSwitchRole(role)`.
- **Online chip:** `@if (network.isOnline()) { Online } @else { Offline Capable }` - a signal read directly in the template.
- **Ionic bits:** `ion-card`, `ion-input` (floating label), `ion-chip`, `ion-note` for validation text.

10.2 DashboardPage (`/dashboard`, authGuard)

- **State:** a single private signal `inspectionsSignal`; everything on screen is a `computed` of it - `totalCount`, `approvedCount`, `pendingWorkerCount` (drafts or pending-sync), `pendingApprovalCount`, `recentInspections` (first 4) and `recentActivity` (flattened audit logs sorted newest-first, top 5).
- **Loading:** `ngOnInit` and `ionViewWillEnter` both call `loadData()` -> `repo.getAll()`; `ion-refresher` triggers `handleRefresh(event)`.
- **Actions:** `switchRole()` (demo swap between Engineer and Supervisor), `logout()`.
- **Navigation:** KPI tiles link to `/inspections?filter=APPROVED` etc. using `routerLink` + `queryParams`. A supervisor sees a shortcut to `/approvals`.
- **Header:** `<app-sync-badge>` shows connectivity and queue counters on most pages.

10.3 InspectionListPage (`/inspections`)

- **Features:** search (`ion-searchbar`, 200 ms debounce), status filter (`ion-segment`), infinite scroll paging (`PAGE_SIZE`), pull-to-refresh, FAB to create.
- **Derived state:** `filteredInspections` -> `pagedInspections` (slice by `displayedCount`) -> `hasMoreItems`.
- **Query params:** `ngOnInit` subscribes to `route.queryParams` and applies `?filter=` coming from the dashboard tiles.
- **Row UI:** condition colour bar, damage flag, status badge (via `statusColor` pipe), and a *sync* badge shown only when `syncStatus !== 'SYNCED'`.
- `loadMore(event)`**:** increases `displayedCount` after a 300 ms timeout, then `event.target.complete()` to hide the spinner.

> **WATCH OUT**
> Important: this page mixes plain fields (`searchQuery`, `selectedFilter`) with `computed()`. Read Chapter 15, item 8, to see why that matters.

10.4 InspectionFormPage (`/inspections/new` and `/inspections/:id/edit`)

The biggest and most instructive page (~700 lines). It serves **both create and edit** mode.

Concern	Implementation
Mode detection	`ngOnInit` subscribes to `route.paramMap`; if `id` exists and is not `'new'` -> `isEditMode = true` and `loadExistingInspection(id)` patches the form with `patchValue`.
Auto-fill from asset id	`onAssetIdChange()` looks the typed id up in `MOCK_ASSETS` and patches name, category and location.
QR scanning	`openQrScanner()` opens `QrScannerModalComponent` in a modal; returns role `'selected'` with `{asset}`.
Conditional validation	`onDamageToggle()` adds/removes `Validators.required` on `damageDescription` and `severity` depending on `hasDamage`; the red box with those fields is shown by `@if (inspectionForm.get('hasDamage')?.value)`.
Photos	`photos` is a `signal<InspectionPhoto[]>`. `capturePhoto()` / `selectPhoto()` call `CameraService`; `previewPhoto()` opens `ImagePreviewModal` (can delete); `removePhoto()` uses `splice` on a copy.
Signature	`<app-signature-pad>` emits a PNG data URL through `(signatureChange)`; stored in `signatureDataUrl`.
Model building	`buildInspectionModel()` merges form values, photos, signature and the current user into an `Inspection` (uppercases `assetId`, ISO date, new id if not editing).
Save Draft	Requires only an asset id -> `repo.saveDraft(model)` -> navigate to `/inspections`.
Submit	`markAllAsTouched()` + `onDamageToggle()`; if the form is invalid show an error toast; otherwise `repo.submit(model)` (online / queued, see 9.4) -> navigate to `/inspections`.

10.5 InspectionDetailsPage (`/inspections/:id`)

- Loads one inspection (`repo.getById`) into a signal in both `ngOnInit` (on route param change) and `ionViewWillEnter`.
- Shows asset info, condition/damage, photos (tap opens read-only preview modal), signature, review comments and the **audit timeline** built from `history`.
- **Contextual footer** (control flow `@if / @else if / @else`): DRAFT -> *Edit & Complete Draft*; supervisor + PENDING_APPROVAL -> *Approve / Reject*; otherwise *Back to list*.
- `confirmApprove()` and `openRejectDialog()` use `AlertController`; rejection **requires** non-empty comments (the handler returns `false` to keep the dialog open).
- `exportReport()` simply calls `window.print()` (browser print dialog).

10.6 ApprovalListPage (`/approvals`, supervisor only)

- Segment tabs **Pending** (`PENDING_APPROVAL`) and **Resolved** (APPROVED or REJECTED), each with a count badge.
- Cards allow quick *View*, *Approve* (optional notes) and *Reject* (mandatory comment) without opening the details page.
- After each action it calls `loadData()` to refresh the list.
- Reached via `roleGuard`; an Engineer who types `/approvals` is redirected to the dashboard with an *Access denied* toast.

11. Shared Components & Pipe

Reusable building blocks in `shared/`.

11.1 SyncBadgeComponent (`<app-sync-badge>`)

- Injects `NetworkService`, `SyncQueueService`, `ActionSheetController`, `ToastService`.
- Template: while `syncQueue.isSyncing()` shows a spinner + *Syncing...*; else a coloured *Online/Offline* button with warning badge `pendingCount()` and danger badge `failedCount() !`.
- Tap opens an **action sheet**: *Sync Queue Now* (blocked with a warning if offline) and *Simulate Offline Mode (Testing)* / *Disable Simulated Offline*.
- Because it is a signal-reading standalone component it can be dropped into any header with a single tag.

11.2 SignaturePadComponent (`<app-signature-pad>`)

Piece	How it works
Inputs/Outputs	`@Input() title`, `@Input() existingSignature`, `@Output() signatureChange = new EventEmitter<string | null>()`.
Canvas access	`@ViewChild('sigCanvas')` + `ngAfterViewInit()` gets the 2D context and sizes the canvas to its wrapper (height 120 px).
Input events	Mouse (`mousedown/move/up/leave`) and Touch (`touchstart/move/end`) listeners. Touch handlers call `preventDefault()` and compute coordinates from `getBoundingClientRect()`; CSS `touch-action: none` stops page scrolling while signing.
Drawing	`startDraw` begins a path (colour `#002b49`, width 2.5, round caps); `draw` adds `lineTo` + `stroke`.
Output	On pointer-up `canvas.toDataURL('image/png')` is emitted; `clear()` wipes the canvas and emits `null`.
Edit mode	If an `existingSignature` is provided and the user has not redrawn, it is shown as an `<img>` overlay.

11.3 QrScannerModalComponent

Displays an **animated viewfinder** (CSS keyframes `scanAnim` moves a laser line) and a list of `MOCK_ASSETS`. Tapping an asset calls `modalCtrl.dismiss({ asset }, 'selected')`. **No real QR decoding happens** - the viewfinder is a simulation for the demo; integrating a scanner plugin is a suggested exercise.

11.4 ImagePreviewModalComponent

Receives `photo` and `canDelete` via `componentProps` (they become `@Input`s). Shows the image full-screen with its timestamp/size, a close button (`dismiss(null,'close')`) and - when `canDelete` is true - a delete button that dismisses with role `'delete'` and `{ photoId }`.

11.5 StatusColorPipe (`{{ value | statusColor:'type' }}`)

Type	Value -> Ionic colour
`status`	APPROVED -> success; PENDING_APPROVAL -> warning; REJECTED -> danger; DRAFT -> medium; else primary
`sync`	SYNCED -> success; PENDING_SYNC -> warning; SYNC_FAILED -> danger; DRAFT -> medium
`condition`	GOOD -> success; FAIR -> primary; POOR -> warning; CRITICAL -> danger
`severity`	LOW -> success; MEDIUM -> warning; HIGH / CRITICAL -> danger
empty/unknown	`medium`

It is a **pure pipe**, so Angular only recomputes it when its inputs change - ideal for badge colours in long lists.

12. End-to-End Scenarios

Follow real user journeys through the code, step by step.

12.1 First launch and sign-in

1. Browser loads `index.html` -> `main.ts` bootstraps -> router sees `''` -> redirect `dashboard`.
2. `authGuard` injects `AuthService`; no session in `localStorage` -> returns `UrlTree('/login')`.
3. `LoginPage` is lazy-loaded. User taps **Field Engineer (Alex Carter)** -> `quickLogin('EMPLOYEE')`.
4. `AuthService.quickSwitchRole` builds a session (fake JWT, +8 h), `persistSession` writes `fia_auth_session` and updates the signal.
5. `router.navigate(['/dashboard'])`; guard now passes; `DashboardPage` loads; repositories seed demo data.

12.2 Create an inspection with damage and submit online

1. Dashboard -> **New Inspection** (`/inspections/new`) -> `InspectionFormPage`.
2. User types `AST-PUMP-001`; `ionChange` -> `onAssetIdChange()` autofills name/location from `MOCK_ASSETS`.
3. Selects **Damage Present** -> `onDamageToggle()` makes severity & description required; the orange box appears.
4. Taps **Take Photo** -> `CameraService.capturePhoto()` -> Capacitor Camera (or web fallback) -> base64 photo pushed into `photos` signal.
5. Draws a signature -> `signatureChange` -> stored as PNG data URL.
6. Taps **Submit** -> form valid -> `buildInspectionModel()` -> `repo.submit(model)`.
7. Online path: audit entry `SUBMITTED`; `remoteRepo.postInspection` (300 ms fake latency) -> saved as `SYNCED` locally; navigate to the list.

12.3 Submit while offline, then reconnect

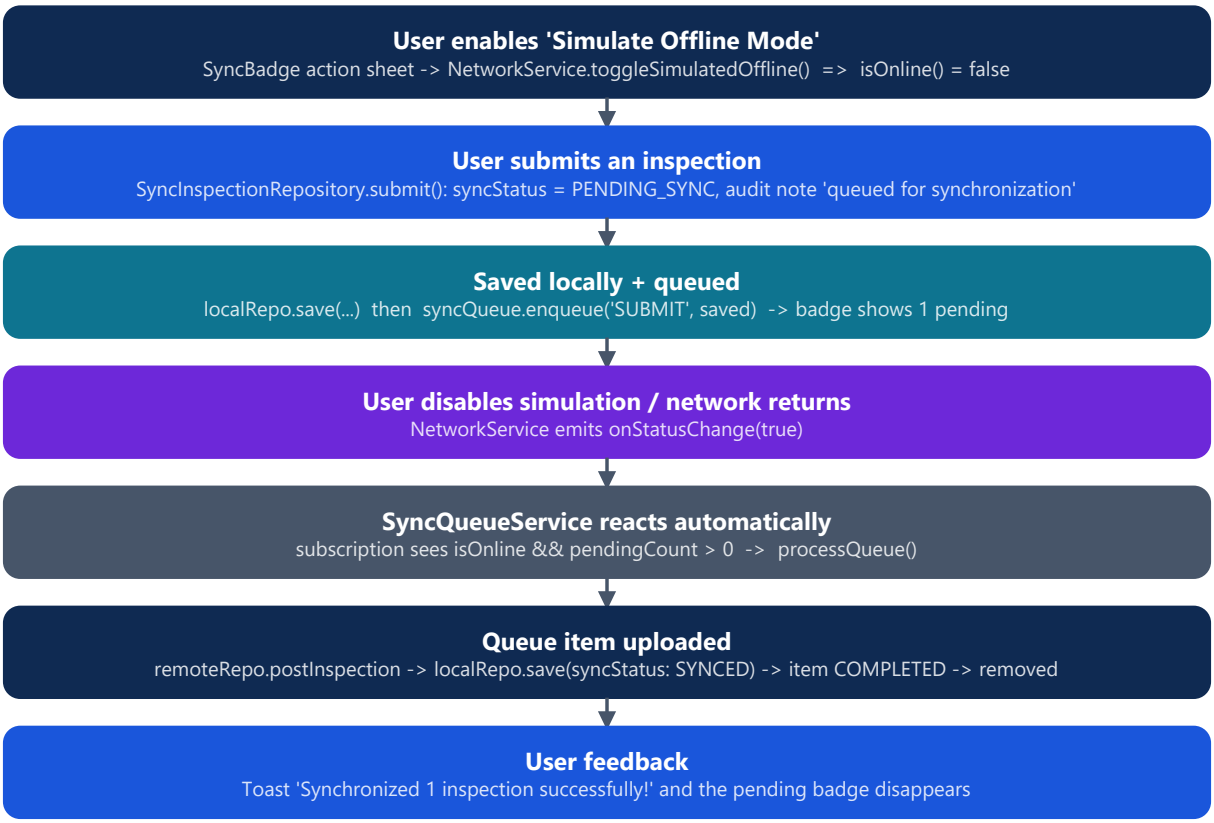


Figure 12.1 - The offline round trip, with no user action needed after reconnecting.

12.4 Supervisor approves or rejects

1. Supervisor signs in (`quickLogin('SUPERVISOR')`) -> dashboard shows a link to **Approvals**.
2. `/approvals`: `authGuard` OK, `roleGuard` compares `route.data['role']` ('SUPERVISOR') with `auth.userRole()` -> allowed.
3. `ApprovalListPage.loadData()` -> `repo.getAll()`; the *Pending* tab filters `status === 'PENDING_APPROVAL'`.
4. **Approve**: AlertController with optional note -> `repo.approve(id, user.id, user.name, note)` -> status APPROVED, audit entry, remote post or queue.
5. **Reject**: comment is mandatory; empty text shows an error toast and the dialog stays open (handler returns `false`).
6. The Engineer later sees the new status, review comments and the audit timeline in the details page.

12.5 Protection scenarios

Situation	What happens	Code
Engineer opens `/approvals`	Warning toast *Access denied: Requires SUPERVISOR permissions*; redirect to dashboard	`roleGuard`
Signed-out user opens `/dashboard`	Redirect to `/login`	`authGuard`
Session older than 8 hours	Persisted session discarded on load; `isAuthenticated()` false; expired toast if detected while running	`loadPersistedSession`, `handleSessionExpired`

Situation	What happens	Code
Unknown URL like `/foo`	Wildcard route redirects to dashboard	`path: '**'`
Server call fails while 'online'	Falls back to queue, warning toast *Network glitch*	`submit()` `catchError`
3 consecutive sync failures	Queue item `FAILED`, inspection `SYNC_FAILED`, error toast, can `retryItem()`	`processQueue`

13. Tooling, Build & Testing

How the project is compiled, run, linted and tested.

13.1 npm scripts

Command	Runs	Result
`npm start`	`ng serve`	Dev server on http://localhost:4200 with hot reload (development configuration)
`npm run build`	`ng build`	Production build (default configuration) into `www/` with hashed file names
`npm run watch`	`ng build --watch --configuration development`	Continuous dev build
`npm test`	`ng test`	Vitest unit tests (watch mode by default; use `--configuration ci` for one-off)
`npm run lint`	`ng lint`	ESLint over `src/**/*.ts` and `src/**/*.html`

13.2 Build configurations

- **development:** no optimisation, source maps, named chunks - easy debugging.
- **production (default for `build`):** minified, output hashing, **budgets** (initial bundle warning 2 MB / error 5 MB; per-component style 10 kB / 20 kB) and `environment.prod.ts` replaces `environment.ts`.
- The builder `@angular/build:application` uses **esbuild + Vite** (fast); lazy routes become separate `chunk-*.js` files.

13.3 Unit testing with Vitest

Tests run in **jsdom** (a simulated browser). `src/test-setup.ts` polyfills `window.matchMedia`, which Ionic components call but jsdom lacks.

Spec file	What it verifies
`auth.service.spec.ts`	Login sets role & authentication; `quickSwitchRole('SUPERVISOR')`; logout clears state and navigates to `/login`.
`sync-inspection.repository.spec.ts`	Draft saved locally with DRAFT status; **offline submit is queued** (`enqueue('SUBMIT', ...)`, `PENDING_SYNC`); approve & reject set status, comments and audit entries.
`inspection.adapter.spec.ts`	DTO -> Domain mapping; Domain -> DTO with full fidelity; Domain <-> local offline entity round trip.
`status-color.pipe.spec.ts`	Colour mapping for status/sync/condition/severity.
`app.component.spec.ts`	Root component smoke test.

Pattern: dependency injection makes the offline logic testable

```
beforeEach(() => {
  mockNetwork = { isOnline: vi.fn(() => true) };            // fake NetworkService
  mockSyncQueue = { enqueue: vi.fn() };                     // spy on calls
  TestBed.configureTestingModule({
    providers: [
      SyncInspectionRepository,                             // the REAL class under test
      { provide: NetworkService, useValue: mockNetwork },   // swapped by DI
      { provide: SyncQueueService, useValue: mockSyncQueue },
      /* ... local/remote/toast mocks ... */
    ],
  });
  repo = TestBed.inject(SyncInspectionRepository);
});

it('should queue inspection when submitting offline', async () => {
  mockNetwork.isOnline.mockReturnValue(false);
  const submitted = await firstValueFrom(repo.submit(testInspection));
  expect(submitted.syncStatus).toBe('PENDING_SYNC');
  expect(mockSyncQueue.enqueue).toHaveBeenCalledWith('SUBMIT', expect.any(Object));
});
```

13.4 Debugging tips

- **DevTools -> Application -> Local Storage:** inspect/edit the four `fia_*` keys to see data and the queue live.
- **DevTools -> Network -> Offline** or the app's *Simulate Offline* switch to test sync.
- **Device toolbar (Ctrl+Shift+M):** emulate a phone to see mobile layout and touch events.
- **Console:** the app logs `Online submission failed, falling back to offline queue` and camera fallback warnings.
- **Android:** `chrome://inspect` to debug the WebView of a connected device. **iOS:** Safari Web Inspector.

14. How to Extend the App

Practical recipes once you understand the structure.

14.1 Add a new page

1. Generate: `npx ng generate @ionic/angular-toolkit:page features/assets/asset-list` (schematics configured for standalone + scss).
2. Add a route in `app.routes.ts` with `loadComponent` and `canActivate: [authGuard]`.
3. Inject `SyncInspectionRepository` or a new service with `inject()`; keep state in signals.
4. Import only the `Ion*` components you use in the component's `imports` array and register icons with `addIcons`.

14.2 Replace localStorage with a real database

Only `LocalInspectionRepository` touches storage. Keep its public methods but switch the internals to `@capacitor/preferences` (small data) or a SQLite plugin / filesystem (inspections + photos). The adapter's `InspectionLocalEntity { id, data, syncStatus, updatedAt }` already looks like a database row.

14.3 Connect a real backend

1. Add `provideHttpClient()` to `main.ts` providers.
2. In `RemoteInspectionRepository` replace the `readDtos/writeDtos` + `delay` simulation with `HttpClient` GET/POST/DELETE calls returning `InspectionApiDto`s; keep using `InspectionAdapter`.
3. Replace `AuthService.login` with a real `POST /login`, store the JWT and attach it with an HTTP interceptor.
4. Handle conflicts (e.g. compare `updatedAt` or use ETags) - the current merge simply prefers server copies of SYNCED items.

14.4 Add native capabilities

Feature	Capacitor option	Hint
Real QR scanning	A barcode-scanner plugin	Replace the simulated viewfinder in `QrScannerModalComponent`; keep `dismiss({asset}, 'selected')`
GPS tag on inspections	`@capacitor/geolocation`	The model/adapter already support `gpsLocation`; set it in `buildInspectionModel()`
Tactile feedback	`@capacitor/haptics` (already installed)	Call `Haptics.impact()` on submit/approve
Status bar / keyboard polish	`status-bar`, `keyboard` (installed)	Configure on `Platform.ready()`
Survive app kill/restart state	`@capacitor/preferences` (installed)	See 14.2

15. Honest Notes & Known Limitations

Things the code does differently from what you might assume - useful when learning or presenting this POC.

These observations come from reading the source. Items marked **(code-read)** were verified by reading the logic and framework semantics; they were **not** executed in a running app for this guide.

1. **README vs code - persistence.** The README mentions SQLite/LocalStorage and `@capacitor/preferences`. The code uses **only** `localStorage`; Preferences is installed but never imported.
2. **No native projects yet.** There is no `android/` or `ios/` folder; only `capacitor.config.ts`. Camera/Network native behaviour is therefore untested here, and the browser fallbacks are what you see.
3. **Mock authentication.** `login()` ignores the password and accepts any email (unknown emails become ad-hoc Employees). Tokens are random strings; there is no server verification.
4. **Simulated backend.** `RemoteInspectionRepository` is another `localStorage` key plus `delay()`. Client and "server" share the same browser storage, so real concurrency conflicts cannot occur.
5. **Simulated QR scanner.** It shows a decorative viewfinder and a pick-list; it never decodes a QR code.
6. **GPS is modelled but never captured.** `gpsLocation` exists in the model and adapter, but the form does not set it (the `location-outline` icon is imported but unused).
7. **Storage size.** Photos and signatures are stored as base64 strings inside JSON in `localStorage` (~5 MB quota) - and each queued item stores a **second** copy of the whole inspection.
8. **Signals + plain fields (code-read).** `InspectionListPage.filteredInspections` is a `computed()` that reads `this.searchQuery` and `this.selectedFilter`, which are ordinary properties. A `computed()` only recomputes when a *signal it read* changes, so typing in the search box or switching the segment does not by itself invalidate it (`onSearchChange` merely sets `displayedCount` to the value it may already have). `ApprovalListPage.displayedInspections` has the same pattern with `activeTab`. The list will refresh when the data signal changes (e.g. pull-to-refresh or re-entering the page). Fix: make these fields signals.
9. **Manual counters.** `pendingCount` / `failedCount` in `SyncQueueService` are plain writable signals updated by hand; they could simply be `computed()` from `queue()`.
10. **Simple retry.** Retries happen immediately in one pass (max 3) with no back-off delay.
11. **Route input binding unused.** `withComponentInputBinding()` is enabled but pages read `ActivatedRoute` directly - both approaches work.
12. **Client-side authorisation only.** Guards hide screens but real authorisation must live on the server.

Suggested fix (not applied - the codebase was left unchanged)

```
// Fix for item 8 - make the filter inputs signals so computed() can track them
public searchQuery   = signal('');
public selectedFilter = signal('ALL');

public readonly filteredInspections = computed(() => {
  let list = this.allInspectionsSignal();
  const q = this.searchQuery().trim().toLowerCase();         // <- now tracked
  if (this.selectedFilter() !== 'ALL') list = list.filter(i => i.status === this.selectedFilter());
  ...
});
// template:  <ion-searchbar [ngModel]="searchQuery()" (ngModelChange)="searchQuery.set($event)">
```

16. Learning Path, Exercises & Cheat Sheet

Turn reading into skill.

16.1 Suggested learning path

Step	Do this	You will learn
1	Run `npm start`, sign in as both roles, create & approve an inspection	The product flow
2	Put a breakpoint in `main.ts`, then in `DashboardPage.loadData()`	Startup order and DI cascade
3	Toggle *Simulate Offline*, submit, watch `fia_sync_queue` in DevTools, go online	Offline-first engine
4	Read `SyncInspectionRepository` next to its spec file	RxJS + testing together
5	Rebuild a small page from scratch (e.g. an Asset list)	Ionic + standalone Angular
6	Add Android with Capacitor and test the real camera	Capacitor workflow

16.2 Exercises (with hints)

#	Exercise	Hint
1	Fix the list filter/search so it updates instantly	Convert `searchQuery`, `selectedFilter` (and Approvals' `activeTab`) to signals - see 15.8
2	Add a *Delete draft* button on the details page	`repo.delete(id)` exists; show an `AlertController` confirmation; navigate back
3	Show queued items in a *Sync Manager* page	`SyncQueueService.queue()` is a signal; use `retryItem(id)`
4	Add exponential back-off to sync retries	Use `setTimeout` or RxJS `retry({ delay })` in `processQueue`
5	Store inspections with `@capacitor/preferences`	Only change `LocalInspectionRepository` internals; keep the adapter
6	Capture GPS when starting an inspection	Install `@capacitor/geolocation`; set `gpsLocation` in `buildInspectionModel()`
7	Write a spec for `SyncQueueService`	Mock `NetworkService.onStatusChange` with a `Subject` and assert `processQueue` is called
8	Add haptic feedback on approve/reject	`import { Haptics } from '@capacitor/haptics'` (already installed)

16.3 "Where do I find...?" cheat sheet

I want to know...	Open this
Where the app starts	`src/index.html` -> `src/main.ts`
Which URL shows which page / guards	`src/app/app.routes.ts`
How login state is kept	`core/auth/auth.service.ts` (`fia_auth_session`)

I want to know...	Open this
How online/offline is detected	`core/services/network.service.ts`
How a submit decides online vs queue	`core/repositories/sync-inspection.repository.ts` -> `submit()`
How the queue retries and syncs	`core/sync/sync-queue.service.ts`
How server data is mapped	`core/repositories/inspection.adapter.ts`
How photos are taken	`core/services/camera.service.ts`
Brand colours / dark mode	`src/theme/variables.scss`, `src/global.scss`
App id / native config	`capacitor.config.ts`
Build output folder and options	`angular.json` (`outputPath.base: www`)

17. Glossary

Plain-language definitions of the terms used in this guide.

Term	Meaning
Angular	Google's TypeScript framework for building web apps (components, DI, routing, forms).
Standalone component	A component that declares its own dependencies; no `NgModule` required.
Bootstrap	Starting Angular and attaching the root component to the page.
Dependency Injection	Framework supplies the objects a class needs instead of the class creating them.
Signal	Reactive value container; reading it inside templates/computed creates automatic dependencies.
computed()	A read-only signal derived from other signals, cached until they change.
Observable / RxJS	Stream of values over time; lazy until subscribed.
Lazy loading	Downloading a feature's code only when first needed (`loadComponent`).
Guard	Function that allows, blocks or redirects a navigation.
Pipe	Template helper that transforms a value (`| statusColor`).
Ionic	UI component library and navigation layer for mobile-style web apps.
Web component / custom element	Browser-native reusable HTML tag such as `<ion-button>`.
Slot	Named placeholder inside a web component where you put child content.
Overlay controller	Service that creates toasts, alerts, modals or action sheets in code.
Capacitor	Native runtime that wraps a web app in a WebView and exposes device APIs via plugins.
Plugin	A package bridging JavaScript to native device features (Camera, Network).
WebView	Embedded browser inside a native app that renders the web UI.
PWA Elements	Web fallbacks for native UIs (camera modal) when running in a browser.
Repository pattern	Abstraction hiding where data is stored/fetched behind a simple interface.
Adapter pattern	Class that converts one data shape into another so layers stay decoupled.
DTO	Data Transfer Object - the shape used on the wire to the server.
Offline-first	Design where the local device copy is primary; the network is used opportunistically.
Sync queue	Persistent list of operations done offline, replayed when online.
Audit trail	Chronological record of who did what and when (`history[]`).
Data URL / base64	Binary data encoded as text: `...`.
Vitest / TestBed	Test runner / Angular's utility for configuring DI in tests.
Zoneless	Angular running without zone.js, relying on signals/events to refresh the UI.

THE WHOLE APP IN ONE LINE

You now have the full map: **index.html -> main.ts -> AppComponent -> router -> guarded lazy page -> repository -> (local | remote | queue) -> storage / Capacitor plugin**. Pick any user action in the app and you should be able to trace it along that path.